# Programmer's Manual

West Virginia University
Group Error 404

**krtc** **4**
int bcd_to_int(unsigned char bcd); 4
int get_seconds(void); 4
int get_minutes(void); 4
int get_hours(void); 4
int get_day(void); 4
int get_month(void); 4
int get_year(void); 4
int int_to_bcd(int value); 4
void set_seconds(int seconds); 5
void set_minutes(int minutes); 5
void set_hours(int hours); 5
void set_day(int day); 5
void set_month(int month); 5
void set_year(int year); 5
**urtc** **6**
define TIME_BUFFER_SIZE 11 6
define DATE_BUFFER_SIZE 11 6
enum Month 6
void get_time(void); 6
int set_time(int hours, int minutes, int seconds); 6
void get_date(void); 6
int set_date(int month, int day, int year); 6
**serial** **7**
int serial_poll(device dev, char *buffer, size_t len) 7
**comhand** **8**
define WHITESPACE_CHARS " \r\n\t" 8
static void mpx_write(const char *s); 8
void cmd_help(void); 8
void cmd_version(void); 8
void comhand(void); 8
**kpcb** **9**
#define PCB_NAME_LENGTH 8 9
#define PCB_NAME_SIZE 1024 9
typedef enum PCB_class 9
typedef enum execution_state 9
typedef enum dispatching_state 9
typedef struct PCB 9
typedef struct PCB_queue 9
extern pcb_queue *ready_queue 9

extern pcb_queue *blocked_queue 9
PCB* pcb_allocate(void); 9
int pcb_free(PCB *pcb); 9
PCB* pcb_setup(const char *name, PCB_class class, int priority); 10
PCB* pcb_find(const char *name); 10
void pcb_insert(PCB *pcb); 10
void pcb_remove(PCB *pcb); 10
static void queue_insert(PCB *pcb, PCB_queue *queue); 10
static void queue_remove(PCB *pcb, PCB_queue *queue); 10
**upcb** **11**
static bool is_valid_name(const char* name); 11
static bool is_unique_name(const char* name); 11
static bool is_valid_class(PCB_class class); 11
static bool is_valid_priority(int priority); 11
static bool is_system_process(PCB *pcb); 11
int create_pcb(const char* name, PCB_class class, int priority); 11
int delete_pcb(const char* name); 11
int set_pcb_priority(const char* name, int priority); 12
int show_pcb(const char* name); 12
void show_ready(void); 12
void show_blocked(void); 12
void show_all(void); 12
int block_pcb(const char* name); 12
int unblock_pcb(const_char* name); 12
int suspend_pcb(const char* name); 13
int resume_pcb(const char* name); 13
**sys_call** **14**
struct context *sys_call(struct context *cont); 14
**loadr3** **15**
void load_R3(void); 15
void load_R3_suspended(void); 15
void load_process(const char* name, dispatching_state dispatch); 15
void resume_all(void); 15
**alarm** **16**
#define ALARM_PROCESS_LIMIT 10 16
#define BUFFER_SIZE 100 16
typedef struct alarm_request 16
bool valid_alarm_time(int hours, int minutes, int seconds); 16
void alarm_process_#(void); 16
int create_alarm_process(int hours, int minutes, int seconds, const char *message); 16
static void build_alarm_name(int slot, char *name); 16

static void initialize_process_context(PCB *pcb, int entry); 16
static int time_to_seconds(int hours, int minutes, int seconds); 16
static void run_alarm_process(int slot); 17
static void (*const alarm_process_entries[ALARM_PROCESS_LIMIT])(void) 17
**mem** **18**
typedef struct MCB 18
typedef struct MCB_list 18
extern MCB_list *free_list 18
extern MCB_list *allocated_list 18
void *initialize_heap(size_t size); 18
void *allocate_memory(size_t size); 18
int free_memory(void *start_address); 18
void allocate_mem(int size); 18
void free_mem(const char* address); 18
void show_allocated_mem(); 19
void show_free_mem(); 19

## krtc

int bcd_to_int(unsigned char bcd);
- Purpose: turns a binary coded decimal into an integer
- Params: *bcd* is an unsigned char representing a binary coded decimal
- Returns: an integer representing an unsigned char representing a binary coded decimal

int get_seconds(void);
- Purpose: gets time from the RTC and returns the current number of seconds
- Params: none
- Returns: an integer representing the current numbers of seconds in the RTC

int get_minutes(void);
- Purpose: gets time from the RTC and returns the current number of minutes
- Params: none
- Returns: an integer representing the current numbers of minutes in the RTC

int get_hours(void);
- Purpose: gets time from the RTC and returns the current number of hours
- Params: none
- Returns: an integer representing the current numbers of hours in the RTC

int get_day(void);
- Purpose: gets time from the RTC and returns the current number of days
- Params: none
- Returns: an integer representing the current numbers of days in the RTC

int get_month(void);
- Purpose: gets time from the RTC and returns the current number of months
- Params: none
- Returns: an integer representing the current numbers of months in the RTC

int get_year(void);
- Purpose: gets time from the RTC and returns the current number of years
- Params: none
- Returns: an integer representing the current numbers of years in the RTC

int int_to_bcd(int value);
- Purpose: turns an integer into a binary coded decimal
- Params: *value* is an integer
- Returns: an unsigned char representing an integer turned into a binary coded decimal

void set_seconds(int seconds);

- Purpose: sets the integer value representing the current seconds in the RTC time
- Params: *seconds* is an integer to be put into the RTC time seconds value
- Returns: none

void set_minutes(int minutes);

- Purpose: sets the integer value representing the current minutes in the RTC time
- Params: *minutes* is an integer to be put into the RTC time minutes value
- Returns: none

void set_hours(int hours);

- Purpose: sets the integer value representing the current hours in the RTC time
- Params: *hours* is an integer to be put into the RTC time hours value
- Returns: none

void set_day(int day);

- Purpose: sets the integer value representing the current days in the RTC time
- Params: *day* is an integer to be put into the RTC time day value
- Returns: none

void set_month(int month);

- Purpose: sets the integer value representing the current months in the RTC time
- Params: *month* is an integer to be put into the RTC time month value
- Returns: none

void set_year(int year);

- Purpose: sets the integer value representing the current years in the RTC time
- Params: *year* is an integer to be put into the RTC time year value
- Returns: none

## urtc

define TIME_BUFFER_SIZE 11
- Purpose: array of size 11 used for get_time return message

define DATE_BUFFER_SIZE 11
- Purpose: array of size 11 used for get_date return message

enum Month
- Purpose: sets months as integers 1-12 for use in set_date method

void get_time(void);
- Purpose: gets the current RTC time and prints it to the comhand's COM1
- Params: none
- Returns: none

int set_time(int hours, int minutes, int seconds);
- Purpose: sets the RTC time for the hours, minutes, and seconds values (for UTC time)
- Params: *hours*, *minutes*, *seconds* are integers that replace the previous relevant values in RTC time
    - *hours* has range 0-23, *minutes* and *seconds* have range 0-59
- Returns: int 0 means the program succeeded; int -1 means an error occurred

void get_date(void);
- Purpose: gets the current RTC date and prints it to the comhand's COM1
- Params: none
- Returns: none

int set_date(int month, int day, int year);
- Purpose: sets the RTC date for the year, month, and day values
- Params: *year*, *month*, *day* are integers that replace the previous relevant values in RTC date
    - *year* has range 0-99, *month* has range 1-12 and *day* has range 1-31
        - *day* param range can vary based off *month* and *year* param range
- Returns: integer 0 if the program succeeded; integer -1 if an error occurred

## serial

int serial_poll(device dev, char *buffer, size_t len)

- Purpose: takes keyboard input using polling technique, stores it to memory, and echos it back to the terminal.
    - Has unique cases for special characters such as Backspace, Escape, etc.
- Params: *dev* is a device figuring out which port to poll from for keyboard input, **buffer* is a char pointer that writes typed characters and stores them to an array (**buffer* is always null-terminated), *len* is used for the buffer size, setting a maximum boundary for the program to prevent overflow
- Returns: positive integer the number of read characters from the buffer, activated through either buffer being full or if the Enter key is pressed; integer 0 if no characters were read, integer -1 if error occurred

## comhand

define WHITESPACE_CHARS " \r\n\t"
- defines what characters need to be split in the string tokenizer

define BUFFER_SIZE 100
- size of the buffer

static void mpx_write(const char *s);
- Purpose: helper function that writes to the console
- Params: constant char pointer *s functions as a string to print to the console
- Returns: none

void cmd_help(void);
- Purpose: lists user help commands
- Params: none
- Returns: none

void cmd_version(void);
- Purpose: shows our current MPX module, compiled build, date and time
- Params: none
- Returns: none

void comhand(void);
- Purpose: handles user commands
- Params: none
- Returns: none

## kpcb

#define PCB_NAME_LENGTH 8
- Purpose: array of size 8 defining max length of PCB name (excluding null terminator)

#define PCB_NAME_SIZE 1024
- Purpose: array of size 1024 defining size of PCB name

typedef enum PCB_class
- Purpose: enumeration block for the PCB class
- Returns: 0 for USER input, 1 for SYSTEM input

typedef enum execution_state
- Purpose: enumeration block for the PCB execution state
- Returns: 0 for READY state, 1 for RUNNING state, 2 for BLOCKED state

typedef enum dispatching_state
- Purpose: enumeration block for the PCB dispatching state
- Returns: 0 for NON_SUSPENDED state, 1 for SUSPENDED state

typedef struct PCB
- Purpose: structure block for the PCB, holds elements used for PCB functions

typedef struct PCB_queue
- Purpose: structure block for the PCB queue, holds PCB queue positions for head & tail

extern pcb_queue *ready_queue
- Purpose: pointer for PCB ready queue

extern pcb_queue *blocked_queue
- Purpose: pointer for PCB blocked queue

PCB* pcb_allocate(void);
- Purpose: PCB/stack memory allocator
- Params: none
- Returns: pointer to the set PCB, null on failure

int pcb_free(PCB *pcb);
- Purpose: frees memory designated for PCB/stack
- Params: *pcb is the pointer to the designated PCB
- Returns: 0 on success, else on failure

PCB* pcb_setup(const char *name, PCB_class class, int priority);

- Purpose: assigns a set name, class, and priority to the designated PCB
- Params: *name is the pointer to the name of the PCB, class is the class of the PCB (USER or SYSTEM), priority is the priority level of the PCB (0-9)
- Returns: a pointer to the updated PCB, null on failure

PCB* pcb_find(const char *name);

- Purpose: name-based PCB locator for both queues
- Params: *name is the pointer to name of the PCB
- Returns: a pointer to the located PCB, null if not found

void pcb_insert(PCB *pcb);

- Purpose: inserts a PCB into a state-and-priority-appropriate queue
- Params: *pcb is a pointer to the PCB to be enqueued
- Returns: none

void pcb_remove(PCB *pcb);

- Purpose: removes a PCB from its queue (doesn't free associated memory or data structures)
- Params: *pcb is a pointer to the PCB to be dequeued
- Returns: 0 on success, else on failure

static void queue_insert(PCB *pcb, PCB_queue *queue);

- Purpose: inserts a PCB into a queue based on priority and number of nodes
- Params: *pcb is a pointer to the PCB, *queue is a pointer to the relevant queue
- Returns: none

static void queue_remove(PCB *pcb, PCB_queue *queue);

- Purpose: removes a PCB from a queue based on position and number of nodes
- Params: *pcb is a pointer to the PCB, *queue is a pointer to the relevant queue
- Returns: none

## upcb

static bool is_valid_name(const char* name);
- Purpose: verifies if PCB name is valid, gives an error message otherwise
- Params: *name is a pointer to the name of the PCB
- Returns: true on valid name, false otherwise

static bool is_unique_name(const char* name);
- Purpose: verifies if PCB name is unique, gives an error message otherwise
- Params: *name is a pointer to the name of the PCB
- Returns: true on unique name, false otherwise

static bool is_valid_class(PCB_class class);
- Purpose: verifies if PCB class is valid, gives an error message otherwise
- Params: class is the class of the PCB
- Returns: true on valid class, false otherwise

static bool is_valid_priority(int priority);
- Purpose: verifies if PCB priority is valid, gives an error message otherwise
- Params: priority is the priority level of the PCB
- Returns: true on valid priority, false otherwise

static bool is_system_process(PCB *pcb);
- Purpose: verifies if the PCB is a system process
- Params: *pcb is a pointer to the PCB
- Returns: true if the PCB is a system process, false otherwise

int create_pcb(const char* name, PCB_class class, int priority);
- Purpose: creates a PCB, then adds it to a designated queue
- Params: *name is the pointer to the name of the PCB, class is the class of the PCB (USER or SYSTEM), priority is the priority level of the PCB (0-9)
- Returns: 0 on success, -1 on invalid params error, -2 on PCB creation fail error

int delete_pcb(const char* name);
- Purpose: removes a PCB from its queue (cannot be a system process*) and frees the respective memory slots afterwards
- Params: *name is a pointer to the name of the PCB
- Returns: 0 on success, -1 on invalid name error, -2 on PCB finding error, *-3 on system process error, else from pcb_remove() or pcb_free()

int set_pcb_priority(const char* name, int priority);

- Purpose: sets the priority level of the designated PCB
- Params: *name is a pointer to the name of the PCB, priority is the updated priority level (0-9)
- Returns: 0 on success, -1 on invalid params error, -2 on PCB finding error, -3 on system process, else on failure of removing from queue

int show_pcb(const char* name);

- Purpose: displays PCB information for a person in the database (name, class, state, suspended status, priority)
- Params: *name is a pointer to the name of the PCB
- Returns: 0 on success, -1 on invalid name error, -2 on lack of name error

void show_ready(void);

- Purpose: shows PCB information held in ready queue
- Params: none
- Returns: none

void show_blocked(void);

- Purpose: shows PCB information held in blocked queue
- Params: none
- Returns: none

void show_all(void);

- Purpose: shows PCB information held in both ready and blocked queues
- Params: none
- Returns: none

int block_pcb(const char* name);

- Purpose: changes the queue state of a PCB from READY to BLOCKED
- Params: *name is a pointer to the name of the PCB
- Returns: 0 on success, -1 on invalid name error, -2 on PCB finding error, -3 on system process, -4 on invalid state error

int unblock_pcb(const_char* name);

- Purpose: changes the queue state of a PCB from BLOCKED to READY
- Params: *name is a pointer to the name of the PCB
- Returns: 0 on success, -1 on invalid name error, -2 on PCB finding error, -3 on invalid state error

int suspend_pcb(const char* name);

- Purpose: changes the suspended status of a PCB from NOT_SUSPENDED to SUSPENDED
- Params: *name is a pointer to the name of the PCB
- Returns: 0 on success, -1 on invalid name error, -2 on PCB finding error, -3 on system process, -4 on invalid state error

int resume_pcb(const char* name);

- Purpose: changes the suspended status of a PCB from SUSPENDED to NOT_SUSPENDED
- Params: *name is a pointer to the name of the PCB
- Returns: 0 on success, -1 on invalid name error, -2 on PCB finding error, -3 on invalid state error

## sys_call

struct context *sys_call(struct context *cont);

- Purpose: swaps the old process and the new process to run and their contexts
- Params: *cont is a pointer to the old context to save
- Returns: a pointer to the  to the new context to be loaded

## loadr3

void load_R3(void);

- Purpose: loads all R3 processes as ready, non-suspended

void load_R3_suspended(void);

- Purpose: load all R3 processes as ready, suspended

void load_process(const char* name, dispatching_state dispatch);

- Purpose: creates a process using the name provided, comparing it to names proc1 - proc5, then loads as a ready process with the given dispatch state
- Params: name is the name of the process, dispatch is the dispatching state of the process

void resume_all(void);

- Purpose: resumes all suspended processes in the ready queue

## alarm

#define ALARM_PROCESS_LIMIT 10
- Purpose: sets the maximum number of available alarm slots

#define BUFFER_SIZE 100
- Purpose: sets the buffer size for the alarms, based off comhand

typedef struct alarm_request
- Purpose: sets the possible values that can be used for alarms

bool valid_alarm_time(int hours, int minutes, int seconds);
- Purpose: checks whether time inputted is valid or not
- Params: hours, minutes, and seconds as integers representing the values for the alarm time
- Returns: bool true if the inputted time is valid, bool false if invalid

void alarm_process_#(void);
- Purpose: given to the pcb context as alarm functions, setting up
- # slots 0 to 9

int create_alarm_process(int hours, int minutes, int seconds, const char *message);
- Purpose: takes given time and message and creates an alarm process
- Params: hours, minutes, and seconds represent specified time, message pointer for the message the alarm will give
- Returns: int 0 if alarm process is successful, int -1 if there are no more slots available or if the PCB fails in creation

static void build_alarm_name(int slot, char *name);
- Purpose: sets the alarm name to be based off the slot number
- Params: slot is the allocated alarm slot number, name pointer to where the name will be allocated

static void initialize_process_context(PCB *pcb, int entry);
- Purpose: creates the process context for an alarm
- Params: pcb pointer to the PCB, entry is the process function to be executed

static int time_to_seconds(int hours, int minutes, int seconds);
- Purpose: converts time to seconds
- Params: hours, minutes, and seconds represent the inputted time
- Returns: int for the total # of seconds in the inputted time

static void run_alarm_process(int slot);

- Purpose: runs an alarm process for the inputted slot
- Params: int slot representing the alarm slot number

static void (*const alarm_process_entries[ALARM_PROCESS_LIMIT])(void)

- Purpose: functions as an array for the function pointers of each alarm slot

## mem

typedef struct MCB
- Purpose: sets the available values for MCB functions

typedef struct MCB_list
- Purpose: sets the head and tail for the MCB functions' lists

extern MCB_list *free_list
- Purpose: pointer for MCB free list

extern MCB_list *allocated_list
- Purpose: pointer for MCB allocated list

void *initialize_heap(size_t size);
- Purpose: Starts the system heap
- Params: size of memory block needed for heap
- Returns: the heap

void *allocate_memory(size_t size);
- Purpose: Validates the size and allocates memory to the first available location with the given size available
- Params: size is the amount of memory space in bytes needed to allocate
- Returns: current pointer to the start address or none

int free_memory(void *start_address);
- Purpose: Validates the hex address and frees the memory from that location based off the given address
- Params: *start_address is the location in the list being freed from memory
- Returns: 0 if successful, -1 error if start address is full or empty

void allocate_mem(int size);
- Purpose: Allocates memory in the heap of the given size; prints out the starting address of the usable memory
- Params: int size is the amount of bytes to allocate
- Returns: error messages if address is NULL or the positive decimal parsing function doesn't equal 0

void free_mem(const char* address);
- Purpose: Frees memory to the heap from the given address (in hex, without the 0x prefix)
- Params: char* address is the start of the usable memory

- Returns: error messages if the address or the hex address parsing function doesn't equal 0

void show_allocated_mem();

- Purpose: prints info for each code block of the allocate_mem list
- Params: none
- Returns: none

void show_free_mem();

- Purpose: prints info for each code block of the free_mem list
- Params: none
- Returns: none